Global Fashion Demand Forecasting and Retail Inventory Optimization Using Apache PySpark

A Big Data Analytics Project Report

Submitted to: Dr. Sonia Mittal

Submitted by:

Dhairya Thacker (25MCA060)
Soham Vaghasia (25MCA064)

Dataset: Global Fashion Retail Stores Dataset
Scale: 6.42 million transactions
Stack: Python 3.13, Apache Spark 3.5.5, MLlib, Flask
Period: January 2023 – March 2025

October 13, 2026

Abstract

This report describes the design and implementation of a distributed analytics pipeline that studies demand behaviour in a multinational fashion retail chain and turns the resulting forecasts into concrete inventory decisions. Working from a public dataset of 6.42 million point-of-sale records covering 35 stores in seven countries, the pipeline performs ingestion, cleaning, integration, exploratory analysis, feature engineering, demand forecasting, and an inventory recommendation stage. Three regression models were trained on Spark MLlib with cross-validated hyperparameter search. A random forest regressor achieved an R^2 of 0.8208 on a held-out chronological test set, beating a naive persistence baseline by roughly 26% in RMSE. The final stage converts forecasts into safety stock levels, reorder points, and per-segment replenishment recommendations. A web dashboard exposes the results for non-technical users. The report discusses dataset limitations openly and suggests paths for further work.

Contents

1 Introduction **2**

2 Background and Related Work **2**

3 Dataset Description **3**

4 Methodology **4**
4.1 Environment and Project Structure 4
4.2 Ingestion 4
4.3 Cleaning 4
4.4 Data Integration 4
4.5 Exploratory Analysis 5
4.6 Feature Engineering and the Forecasting Grain 5
4.7 Forecasting Models 6
4.8 Inventory Recommendation Engine 6
4.9 Deployment 7

5 Results **7**
5.1 Model Performance 7
5.2 Feature Importance 7
5.3 Segment-Level Analysis 8

6 Inventory Recommendations **9**

7 Engineering Notes **9**

8 Discussion and Limitations **10**

9 Conclusion and Future Work **10**

1 Introduction

Retail chains live and die by their inventory decisions. Order too little and shelves empty out, customers leave, and revenue disappears. Order too much and capital sits in warehouses, markdowns pile up, and margins erode. For a company operating dozens of stores across multiple countries, these decisions get harder, because demand behaves differently in each market, each product category responds differently to promotions, and the sheer volume of transaction data makes conventional spreadsheet-based analysis impractical.

This project examines that problem through the lens of distributed computing. The dataset we work with contains 6.42 million transaction records from a global fashion retailer with stores in the United States, China, Germany, France, Spain, Portugal, and the United Kingdom. Loading a table of that size into memory with standard tools is possible on a workstation but fragile, and the joins, window functions, and aggregations we need would be slow. Apache Spark was built precisely for this kind of workload, and using PySpark lets us demonstrate distributed processing on a laptop while keeping the same code portable to a cluster.

The goal is not simply to produce another sales dashboard. Plenty of those exist. What we set out to build is a pipeline that goes one step further: from raw transactions all the way to an actionable recommendation such as "store 6 in Shanghai should order 744 units of the Children category within the week." That last step, translating a forecast into a decision, is where most student analytics projects stop short, and it is the part we consider most valuable.

Three questions guided the work:

1. How does demand vary across countries, stores, categories, and time, and what patterns emerge at each level of aggregation?
2. Can historical demand, pricing, and discount information predict future demand well enough to be useful, and which model family performs best?
3. How can those forecasts be converted into inventory decisions that a retail planner could actually execute?

The rest of this report is organised as follows. Section 2 reviews related work briefly. Section 3 describes the dataset. Section 4 walks through the methodology, phase by phase. Section 5 presents model results, and Section 6 covers the inventory recommendation engine. Section 4.9 explains the dashboard, Section 8 discusses limitations honestly, and Section 9 wraps up.

2 Background and Related Work

Demand forecasting has a long history in operations research, and the retail setting in particular has attracted attention for decades. Classical approaches such as ARIMA and exponential smoothing remain in wide use because they are interpretable and cheap to compute. Their weakness in our context is that they model each time series independently; with 105 store–category combinations, fitting and maintaining that many separate models becomes cumbersome, and they cannot easily incorporate exogenous features like discount percentage.

More recent work has moved toward machine learning. Regression models trained on lagged demand, calendar features, and promotional signals can share statistical strength across series and typically outperform univariate baselines when enough history exists. Gradient boosting and random forests are the workhorses here, and both are available in Spark MLlib, which makes them natural choices for a distributed pipeline. Deep learning approaches such as temporal fusion transformers show promise but demand far more data and compute than this project requires.

Discount effects on demand have been studied extensively in marketing literature, and the general finding, that promotions lift unit volume while reducing per-transaction revenue, reappears in our data. We do not attempt to build a price elasticity model, but we include discount features in the forecasting stage and examine aggregate discount effects during exploratory analysis.

On the systems side, Apache Spark is now the default tool for large-scale tabular analytics. The DataFrame API, window functions, and MLlib pipelines are mature, and the code we wrote could be moved to a cluster without modification. We treat the use of Spark as both a practical choice and a demonstration: a key objective of the project is to show that the processing genuinely uses distributed operations rather than treating PySpark as a fancy file reader.

One characteristic of the dataset deserves mention up front. The data is synthetic, generated to simulate a retail chain rather than scraped from real stores. A public analysis of the same dataset reports deterministic return labels and perfect classification scores for return prediction, which is a strong signal that some patterns are artificially clean. We therefore treat absolute performance numbers as internal comparisons between models rather than as claims about real-world accuracy.

3 Dataset Description

The Global Fashion Retail Stores Dataset, published on Kaggle by user ricgomes, consists of six CSV files. The main fact table is `transactions.csv`, which holds 6,416,827 records and occupies about 805 MB on disk. Each record represents a single line item: an invoice may span several rows distinguished by a line number. The remaining five files are dimensional tables.

Table 1: Overview of the six source tables.

File	Rows	Role
`transactions.csv`	6,416,827	Fact table, one row per invoice line
`customers.csv`	1,643,306	Customer demographics
`products.csv`	17,940	Product catalogue and costs
`stores.csv`	35	Store locations and geography
`employees.csv`	404	Staff assignments
`discounts.csv`	181	Promotional campaign periods

Transaction rows carry a mixture of identifiers (invoice, customer, product, store, employee), descriptive fields (size, colour, SKU, currency), and quantitative fields (unit price, quantity, discount fraction, line total, invoice total). The date column is a timestamp, which allows hour-level analysis in addition to daily aggregation. A `Transaction Type` field distinguishes sales from returns, and a payment method field records how each sale settled.

The product table groups items into three top-level categories: Feminine, Masculine, and Children. Within each category there are up to eleven sub-categories such as Coats and Blazers, Pants and Jeans, and Sportswear. Descriptions are provided in six languages, and a production cost column enables gross-margin style analysis though we did not use it heavily. The store table contains latitude and longitude coordinates, which we did not exploit but which would support geographic visualisation in future work.

Discount campaigns are stored separately. Each row in `discounts.csv` specifies a date range, a discount percentage, and a target category–subcategory pair. This structure means the discount campaign cannot be joined on a simple key; the join condition has to test whether each transaction's date falls within a campaign window for the same category. We handle this with a compound join, described in Section 4.4.

The data spans January 2023 through March 2025, roughly 27 months. That is short

for capturing multi-year seasonality, but it is enough to observe an annual cycle if we look at each calendar month across both full years. The synthetic nature of the data shows in a few places: several stores in China record dramatically higher average transaction values than their European counterparts (around $349 versus $28–45), which seems implausibly large even accounting for currency and market differences.

4 Methodology

The pipeline is organised into numbered phases that run sequentially. This section walks through each one, explaining what it does and, more importantly, why it does it that way.

4.1 Environment and Project Structure

The project runs on Python 3.13 with PySpark 3.5.5 and Java 8. The Spark session is configured for local execution using all available cores, with six gigabytes allocated to the driver and the shuffle partition count reduced from its default of 200 to a small number that suits a single machine. On a workstation, the default 200 shuffle partitions would create thousands of tiny files and waste time on scheduler overhead, so we set it to four.

The codebase separates concerns into modules under `src/`: configuration constants, ingestion helpers, cleaning functions, feature engineering utilities, model builders, and evaluation routines. The main pipeline script imports these modules and orchestrates the phases. Outputs are written to a structured directory tree with separate folders for raw data, Parquet products, model artefacts, analysis tables, and charts. Keeping raw files immutable matters: every transformation reads from `data/raw/` and writes elsewhere, so the pipeline is reproducible from scratch.

4.2 Ingestion

Ingestion is deliberately simple. Each CSV is read with header and schema inference enabled, producing six Spark DataFrames. On the transactions file the read takes about 16 seconds and the count returns 6,416,827 rows, matching the documented figure. Inferring schemas rather than declaring them explicitly is a trade-off: it is convenient during exploration, but for production one would declare types up front to avoid a second pass over the data. We record baseline counts for every table before any transformation so that later row-count changes can be attributed to specific steps.

4.3 Cleaning

The cleaning phase standardises column names to lowercase with underscores, a small step that prevents a class of bugs later when referencing columns programmatically. Timestamps in the transactions table arrive as strings in `yyyy-MM-dd HH:mm:ss` format and are parsed accordingly. Numeric columns are cast to double or integer as appropriate, and string fields are trimmed of whitespace.

Duplicate handling then removes 22,312 transaction rows that share both an invoice identifier and line number. That the dataset ships with duplicates at all is worth noting; it suggests the simulation generated some invoice lines twice. After removing rows with null keys in the product, store, or invoice fields, 6,394,515 records remain, a loss of 0.35% from the original count.

4.4 Data Integration

The integration phase joins the fact table to the dimension tables using four sequential left joins. The order matters and so does the column selection. Joining stores first adds country and city, which we need for nearly every downstream analysis. Joining customers next is a little

delicate: the customer table also contains city and country columns, which would collide with the store-derived columns and make later references ambiguous. We therefore drop those two columns from the customer side before joining, keeping only genuinely additive fields such as name, gender, and job title.

The product join is straightforward. The discount join is not. Since campaigns are defined by a category and a date window, the join condition combines three predicates: the transaction category matches the campaign category, the sub-category matches, and the transaction date falls between the campaign's start and end dates. Written in PySpark this is a join on a boolean expression rather than a key. After this join we coalesce the discount column: where the transaction itself carries a discount value we keep it, and where it does not we fall back to the campaign percentage. Row counts hold steady at every join, confirming the left-join semantics behaved as intended.

4.5 Exploratory Analysis

Before building anything predictive, we ran nine aggregations to understand the shape of the data. These cover revenue and transaction counts by country, by store, and by category; revenue and units by month; a finer breakdown by sub-category; demand by day of week and by hour of day; and a comparison of discounted versus full-price transactions. A tenth analysis cross-tabulates country against category.

Several findings stood out. China accounts for roughly 73% of global revenue despite holding only five of the thirty-five stores, driven by that country's high average transaction values. Revenue follows a clear seasonal rhythm with December far ahead of other months; December 2024 alone logged over 800,000 units against a monthly baseline closer to 150–350 thousand in slower periods. Weekend days see roughly twice the traffic of weekdays, and shopping concentrates in the late afternoon and early evening. Discounted transactions carry an average line value about 37% lower than full-price ones, consistent with the standard promotional trade-off.

One anomaly deserves mention because we did not fully resolve it. Transactions recorded at hour zero show negative aggregate revenue, around minus 40 million. The most plausible explanation is that returns are timestamped at midnight, and the negative line totals from returns outweigh any sales recorded in that hour. We left these records in place for the demand analysis, since quantities remain positive, but flag the pattern here as an artifact worth investigating in any follow-up work.

4.6 Feature Engineering and the Forecasting Grain

Choosing the right aggregation grain for forecasting took some thought. The most detailed option, one series per store per product per day, produced 5.7 million rows and extreme sparsity; most product–store combinations sell only intermittently, so a daily series is mostly zeros, and the 16,198 distinct product identifiers also exceed the default bin limit of tree-based Spark models. A coarser grain of store by category by day yields 686,776 rows across 105 active segments. That is dense enough for lag features to be meaningful and fine enough for store-level inventory decisions.

On top of the aggregated demand table we build three families of features. Temporal features extract year, month, quarter, day of week, and week of year from the date. Pricing features summarise the average unit price and average discount per segment-day, along with a binary flag for whether any discount applied. Historical features use window functions partitioned by store and category: lagged demand at one, seven, fourteen, and thirty days, plus rolling averages over seven, fourteen, and thirty days. The rolling averages turn out to carry most of the predictive signal, as the feature importance results will show.

Windows are ordered by date within each partition, and Spark computes them in a single distributed pass. This is one of the places where PySpark justifies itself: the equivalent in

Pandas would require sorting six million rows and looping over groups, which is slower and cannot scale beyond memory.

4.7 Forecasting Models

The demand data is split chronologically, not randomly. Training uses the first 70% of dates, validation the next 15%, and testing the final 15%. This matters. A random split would scatter future observations into the training set and leak information backward in time, producing optimistic metrics that collapse in deployment. Machine learning courses often warn about this and it is worth repeating here because the temptation to shuffle is strong when the code is simpler that way.

Three model families were trained, all through Spark MLlib pipelines that chain categorical indexing, feature vector assembly, and the estimator itself:

- **Random forest**, tuned over tree counts of 20 and 30 and depths of 6, 8, and 10, with three-fold cross-validation.
- **Gradient boosted trees**, tuned over 15 and 20 iterations and depths of 4 and 5, with two-fold cross-validation. Two folds rather than three because boosting trains sequentially and the runtime adds up quickly on a laptop.
- **Linear regression** with L2 regularisation, used as a simple benchmark against the tree ensembles.

A naive persistence baseline was also computed, where the forecast for tomorrow equals today's demand. Baselines like this keep us honest; if a machine learning model cannot beat persistence, it is not earning its complexity.

4.8 Inventory Recommendation Engine

The final stage turns forecasts into decisions. For each store–category segment we compute the following quantities:

$$\text{Safety stock} = z \cdot \sigma_d \cdot \sqrt{L} \tag{1}$$

$$\text{Reorder point} = \bar{d} \cdot L + \text{Safety stock} \tag{2}$$

$$\text{Required inventory} = \bar{d} \cdot H + \text{Safety stock} \tag{3}$$

Here σ_d is the standard deviation of daily demand, L is the replenishment lead time (set to seven days), H is the planning horizon (thirty days), and z is the service-level factor. Setting $z = 1.65$ corresponds to a 95% service level, meaning stock should cover demand on 95% of days.

Because the dataset contains no inventory snapshots, current stock levels are simulated from historical patterns: a segment is assigned roughly ten days of average demand, modulated by a deterministic factor derived from the store identifier so that some stores start better stocked than others. This is a limitation we state plainly. The logic of the engine, however, is independent of whether stock levels come from simulation or a warehouse system; swapping in real data is a matter of replacing one column.

Each segment is then classified. If simulated coverage falls below roughly 70% of the lead time, the segment is flagged as a stockout risk. If coverage exceeds the planning horizon by 30% or more, it is an overstock risk. Anything between is healthy. Stockout segments receive an order recommendation equal to the gap between required and current stock, and each recommendation carries a severity level from critical to low based on how thin coverage is.

4.9 Deployment

Results are exposed through a small Flask application backed by a set of JSON endpoints that read the pipeline's output files. Four pages are provided: an overview with headline metrics and charts, an inventory page centred on the recommendation table with filtering and search, an insights page with four tabs covering geography, seasonality, discounts, and customer economics, and a models page documenting evaluation results. The interface follows a light theme with a restrained palette, and charts are rendered as static images produced by Matplotlib during the pipeline run. Reading precomputed files rather than querying a database keeps the deployment simple, which suits a project of this scope.

5 Results

5.1 Model Performance

Table 2 summarises test-set performance. The random forest comes out ahead on every metric that matters. Its RMSE of 9.54 units per segment-day compares favourably with the naive baseline's 12.87, a reduction of about 26%. The R^2 of 0.8208 means the model explains roughly 82% of the variance in held-out demand, which for retail data of this granularity is respectable.

Table 2: Test-set performance of the four forecasting approaches.

Model	RMSE	MAE	R^2	sMAPE
Naive baseline (persistence)	12.87	6.24	—	—
Linear regression	9.58	4.84	0.8194	47.7%
Random forest	**9.54**	**4.43**	**0.8208**	39.9%
Gradient boosted trees	9.96	4.52	0.8045	40.6%

Two results deserve comment. First, linear regression performed nearly as well as the random forest, with an R^2 of 0.8194 against the forest's 0.8208. That is a narrow margin. It suggests that most of the predictable signal in this dataset is captured by fairly simple relationships, particularly the rolling average of recent demand. When a linear model with seventeen features closes in on a 30-tree ensemble, the extra complexity buys surprisingly little.

Second, gradient boosting underperformed both of its competitors. That was unexpected. Boosting usually edges out random forests on tabular data, and the likely explanation here is the deliberately small hyperparameter grid. We capped boosting at 20 iterations and depth 5 to keep laptop runtimes reasonable, and that may simply be too few iterations for the model to find its footing. More iterations and a finer learning rate search would probably change this ranking, and we note it as a limitation of the experiment rather than a property of the algorithm.

The sMAPE figures deserve a caveat of their own. Values in the 40% range look alarming next to the R^2, but the metric penalises errors on low-demand segments heavily: when actual demand is 2 units and the model predicts 4, the percentage error is large even though the absolute error is trivial. With a mean daily demand around 10 units per segment, an MAE of 4.4 translates to typical errors in the tens of percent at the segment level. The R^2 and RMSE tell a fairer story here, which is why we report all four metrics together rather than picking the most flattering one.

5.2 Feature Importance

Feature importance rankings, extracted from the fitted forest, are shown in Table 3. The results are unambiguous: the seven-day rolling average alone accounts for 35% of importance,

and combined with the fourteen-day average the two rolling features carry 56%. The model has essentially learned that recent momentum is the best guide to near-term demand, which aligns with intuition and with the broader forecasting literature.

Table 3: Top features by importance in the random forest model.

Rank	Feature	Importance
1	Rolling average, 7 days	0.350
2	Rolling average, 14 days	0.211
3	Average discount	0.108
4	Discount flag	0.075
5	Lag 1 day	0.053
6	Rolling average, 30 days	0.050
7	Store identifier	0.028
8	Day of week	0.027
9	Month	0.018
10	Average unit price	0.018

Discount features rank third and fourth, which is notable given that discounts apply to a minority of segment-days. Calendar features (day of week, month) contribute modestly but non-trivially, consistent with the weekly and seasonal patterns visible in the exploratory analysis. The near-zero importance of the year feature is unsurprising since the dataset spans only two full years.

5.3 Segment-Level Analysis

Aggregate metrics can hide important variation, so we evaluated the best model separately by country and by category. Table 4 presents the country breakdown.

Table 4: Random forest performance by country, sorted by R^2.

Country	RMSE	MAE	R^2	Test rows
United Kingdom	6.52	3.16	0.8303	14,190
China	13.47	6.74	0.8192	15,318
Germany	8.54	3.81	0.8176	14,768
Spain	6.46	3.21	0.7953	14,245
United States	14.50	7.31	0.7910	15,571
France	6.21	3.35	0.7889	14,881
Portugal	5.78	3.06	0.7690	14,044

The spread is meaningful. The United Kingdom is the most predictable market at an R^2 of 0.83, while Portugal trails at 0.77. China and the United States show the largest absolute errors, which makes sense because their stores move the most units; an RMSE of 13–15 on a series averaging 20–30 units daily is no worse in relative terms than an RMSE of 6 on a series averaging 10. This per-segment view is something a single global score would have obscured, which is exactly why we computed it.

Category results follow a similar pattern. Feminine leads with an R^2 of 0.8287, followed by Masculine at 0.8086 and Children at 0.7863. Children has the smallest absolute errors (RMSE 5.25) simply because volumes are lower, but proportionally it is the hardest to predict, possibly because children's wear demand depends on growth spurts and school calendars that the feature set does not capture.

6 Inventory Recommendations

The recommendation engine processes all 105 store–category segments and produces the outcome summarised in Table 5.

Table 5: Inventory risk summary across 105 store–category segments.

Status	Segments
Stockout risk	14 (13.3%)
Overstock risk	0 (0.0%)
Healthy	91 (86.7%)
Total recommended order value	$3,198,779
Lead time assumption	7 days
Service level	95%

Several patterns in the recommendations are worth pulling out. Children's wear dominates the risk list: twelve of the fourteen stockout flags belong to that category. This lines up neatly with the modelling results, where Children was the least predictable category and the one where actual demand most often exceeded the model's expectation. A forecast that systematically under-predicts spikes will, downstream, produce systematic urgency in the inventory engine.

The New York store stands out at the location level, with all three of its categories flagged. Its Children segment shows only 2.8 days of coverage against a seven-day lead time, which in a real operation would justify an emergency order or a transfer from another store. Marseille showed a similar pattern on two of three categories, and since nothing in the analytical setup singles out that store, the flag is either a genuine demand spike in the final weeks of the dataset or an artifact of the simulation. We cannot distinguish between those possibilities with the information available, and say so.

The feature importance results and the inventory outputs tell a coherent story together. Segments with volatile recent demand produce wider rolling averages and larger standard deviations, which inflate safety stock requirements and make coverage thresholds harder to satisfy. The engine is, in effect, charging a higher buffer to unpredictability, which is the correct behaviour for a service-level-based policy.

No overstock risks were detected, which is a direct consequence of the stock simulation. Since simulated inventory is tied to each segment's own historical average demand, systematically over-stocked segments cannot arise under that construction. With real inventory data, the overstock logic would activate and the balance of the two risk types would shift. We consider the stockout machinery the substantive contribution here, with overstock detection demonstrated but untested against real conditions.

7 Engineering Notes

A few implementation decisions are worth recording for anyone extending this work.

Window functions drove the feature engineering. Computing lagged and rolling demand requires partitioning by store and category and ordering within each partition by date, which Spark handles natively and in parallel. Two subtleties caused trouble and are worth flagging. First, date arithmetic matters: computing a rolling window over seven rows is not the same as computing it over seven days when a segment has missing days, and our data does have gaps. We chose row-based windows for simplicity and note the distinction. Second, the boundary of each partition produces null lags for the first rows of the series; these are filled with zeros, which is a modelling convenience rather than a statement that demand was zero.

Disk pressure shaped the tuning phase. Cross-validating the random forest over six parameter combinations with three folds means eighteen model fits, and Spark's shuffle writes filled the temporary directory on a laptop with limited free space early in development. Reducing the shuffle partition count, pointing the Spark local directory at a controlled location, and caching the training DataFrame cut the disk footprint enough to complete the run. The lesson, unglamorous but real, is that distributed frameworks still run on finite disks.

Evaluation code went through a similar simplification. The symmetric MAPE calculation originally collected per-row errors through a forEach loop, which was slow and fragile at scale. Rewriting it as a column expression evaluated in a single aggregation pass made it fast and eliminated a class of executor-side failures. Small changes like this are where most of the debugging time went.

8 Discussion and Limitations

Honesty about limitations serves the reader better than a polished narrative of success, so this section is deliberately blunt.

The dataset is synthetic. Simulated retail data cannot reproduce the messiness of real transactions: no stockouts, no substitution effects, no competitor pricing moves, no weather. Some artefacts make this plain, including the midnight return anomaly and the implausibly high Chinese transaction values. Every number in this report should be read as an internal comparison, not as an estimate of real-world performance.

The forecasting grain trades detail for density. Store–product–day would be more actionable for inventory planning but too sparse for the models as configured; store–category–day is dense enough to learn from but forces single order decisions across whole product ranges. A production system might forecast at product level using hierarchical reconciliation, where product forecasts are constrained to sum to the category-level prediction. That is a substantial extension and beyond what we attempted.

Safety stock is treated as static. In practice, forecast uncertainty fluctuates through the year, and Christmas-week demand is not as predictable as an ordinary Tuesday in March. A dynamic safety stock that widens during promotional periods would be more realistic.

The inventory simulation is the weakest link in the chain. Real stock levels would let the engine detect genuine overstock, validate the stockout flags against actual sell-through, and measure how much the recommendations would have saved. Without them, the engine demonstrates methodology rather than delivered decision value.

Finally, the models use no external data. Promotions are captured only through the transactional discount field, and store-level foot traffic, local events, holidays, and economic conditions are absent. The feature set could absorb such signals easily if they were available.

9 Conclusion and Future Work

This project set out to connect large-scale transaction processing with retail decision-making, and the pipeline we built walks that path end to end: from six raw CSV files to a ranked list of replenishment orders, with a forecasting model in the middle that beats a naive baseline by a comfortable margin. Along the way the work exercised the parts of PySpark that separate it from plain Pandas: distributed joins, window functions over partitioned data, MLlib pipelines with cross-validated tuning, and Parquet as a storage layer.

The results that matter most are these. A random forest regressor explains roughly 82% of demand variance at the store–category–day level, with rolling recent-demand averages carrying over half the predictive weight. Discounts matter substantially as predictors, more than calendar effects. Forecast quality varies across markets from R^2 0.77 in Portugal to 0.83 in the United Kingdom, a spread that a single global metric would have hidden. And the derived inventory

engine flags 14 of 105 segments as stockout risks, concentrated in children's wear and in the New York store, with a total recommended order value of roughly $3.2 million.

Looking forward, the obvious next steps are external data enrichment, hierarchical forecasting at product level, dynamic safety stock, and validating the inventory logic against real stock records. On the engineering side, the pipeline would move from local mode to a small cluster without code changes, and the dashboard would gain real-time endpoints in place of precomputed files. Any of these would be a meaningful project in its own right.

References

[1] R. Gomes. *Global Fashion Retail Stores Dataset*. Kaggle, 2025. Available: `https://www.kaggle.com/datasets/ricgomes/global-fashion-retail-stores-dataset`

[2] M. Zaharia, R. S. Xin, P. Wendell, T. Das, M. Armbrust, A. Dave, X. Meng, J. Rosen, S. Venkataraman, M. J. Franklin, A. Ghodsi, J. Gonzalez, S. Shenker, and I. Stoica. "Apache Spark: A Unified Engine for Big Data Processing." *Communications of the ACM*, vol. 59, no. 11, pp. 56–65, 2016.

[3] X. Meng, J. Bradley, B. Yavuz, E. Sparks, S. Venkataraman, D. Liu, J. Freeman, D. Tsai, M. Amde, S. Owen, D. Xin, R. Xin, M. Franklin, R. Zadeh, M. Zaharia, and A. Talwalkar. "MLlib: Machine Learning in Apache Spark." *Journal of Machine Learning Research*, vol. 17, no. 34, pp. 1–7, 2016.

[4] L. Breiman. "Random Forests." *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001.

[5] J. H. Friedman. "Greedy Function Approximation: A Gradient Boosting Machine." *Annals of Statistics*, vol. 29, no. 5, pp. 1189–1232, 2001.

[6] S. Makridakis, E. Spiliotis, and V. Assimakopoulos. "The M4 Competition: 100,000 Time Series and 61 Forecasting Methods." *International Journal of Forecasting*, vol. 36, no. 1, pp. 54–74, 2020.

[7] N. Huber and A. Stuckenschmidt. "Daily Retail Demand Forecasting Using Machine Learning." *International Journal of Forecasting*, vol. 36, no. 4, pp. 1424–1440, 2020.